



МИНОБРНАУКИ РОССИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

«МИРЭА – Российский технологический университет»

РТУ МИРЭА

**Институт информационных технологий (ИИТ)
Кафедра прикладной математики (ПМ)**

ОТЧЕТ ПО ПРАКТИЧЕСКОЙ РАБОТЕ №5
по дисциплине «Системы анализа больших данных»

Студент группы *ИНБО-21-23. Илюхин К.Н.*

(подпись)

Преподаватель *Алексеева Е.С.*

(подпись)

Москва 2026 г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ.....	2
1 ВВЕДЕНИЕ.....	4
1.1 Цель и постановка задачи.....	4
2 ХОД РАБОТЫ	5
ВЫВОДЫ	16

1 ВВЕДЕНИЕ

1.1 Цель и постановка задачи

Целью практической работы является формирование навыков оркестрации распределённых вычислений с применением Apache Airflow на примере запуска PySpark-приложения на кластере Arenadata Hyperwave и загрузки результатов в аналитическую СУБД.

В рамках работы были выполнены следующие задачи:

- развёртывание Apache Airflow в Docker Compose;
- настройка SSH-подключения Airflow к узлу кластера ADH;
- настройка PostgreSQL-подключения Airflow к аналитической СУБД;
- подготовка shell-скриптов и PySpark-приложения на узле ADH;
- создание DAG для оркестрации процесса обработки данных;
- запуск DAG и проверка результатов;
- загрузка аналитических витрин в PostgreSQL.

2 ХОД РАБОТЫ

Перед развёртыванием Airflow была проверена версия Docker и Docker Compose на ВМ ubuntu-cass.

```
200user@adcm:~$ docker --version
Docker version 29.3.0, build 5927d80
user@adcm:~$ docker compose version
Docker Compose version v5.1.1
user@adcm:~$
```

Рисунок 1 — вывод `docker --version` и `docker compose version`

```
user@adcm:~$ free -h
              total        used        free      shared  buff/cache   available
Mem:           1.9Gi       590Mi       460Mi       1.0Mi       912Mi       1.2Gi
Swap:          2.0Gi          0B        2.0Gi
user@adcm:~$ df -h /
Filesystem      Size  Used Avail Use% Mounted on
/dev/sda2       25G   11G   13G   46% /
user@adcm:~$
```

Рисунок 2 — вывод `free -h` и `df -h /`

Была создана рабочая директория `airflow-practice-05` с подкаталогами `dags`, `logs`, `plugins`, `config`.

```
kirill@ubuntu-cass:~$ ss -tlnp | grep 8080
kirill@ubuntu-cass:~$ mkdir -p ~/airflow-practice-05
kirill@ubuntu-cass:~$ cd ~/airflow-practice-05
kirill@ubuntu-cass:~/airflow-practice-05$ mkdir -p ./dags ./logs ./plugins ./config
kirill@ubuntu-cass:~/airflow-practice-05$ ls -la ~/airflow-practice-05
total 24
drwxrwxr-x 6 kirill kirill 4096 Sep 30 23:12 .
drwxr-x--- 6 kirill kirill 4096 Sep 30 23:11 ..
drwxrwxr-x 2 kirill kirill 4096 Sep 30 23:12 config
drwxrwxr-x 2 kirill kirill 4096 Sep 30 23:12 dags
drwxrwxr-x 2 kirill kirill 4096 Sep 30 23:12 logs
drwxrwxr-x 2 kirill kirill 4096 Sep 30 23:12 plugins
kirill@ubuntu-cass:~/airflow-practice-05$
```

Рисунок 3 — вывод `ls -la ~/airflow-practice-05`

Официальный `docker-compose.yaml` был загружен с сайта Apache Airflow (версия 3.2.1).

```

kirill@ubuntu-cass:~/airflow-practice-05$ curl -Lfo 'https://airflow.apache.org/docs/apache-airflow/3.2.1/docker-compose
.yaml'
% Total    % Received % Xferd  Average Speed   Time    Time     Time  Current
           Dload  Upload   Total     Spent    Left     Speed
100 12983  100 12983    0     0    95198      0 --:--:-- --:--:-- --:--:-- 95463
kirill@ubuntu-cass:~/airflow-practice-05$ ls -la
total 40
drwxrwxr-x 6 kirill kirill 4096 Sep 30 23:12 .
drwxr-x--- 6 kirill kirill 4096 Sep 30 23:11 ..
drwxrwxr-x 2 kirill kirill 4096 Sep 30 23:12 config
drwxrwxr-x 2 kirill kirill 4096 Sep 30 23:12 dags
-rw-rw-r-- 1 kirill kirill 12983 Sep 30 23:12 docker-compose.yaml
drwxrwxr-x 2 kirill kirill 4096 Sep 30 23:12 logs
drwxrwxr-x 2 kirill kirill 4096 Sep 30 23:12 plugins
kirill@ubuntu-cass:~/airflow-practice-05$

```

Рисунок 4 — вывод ls -la с файлом docker-compose.yaml

Файл .env содержит AIRFLOW_UID и FERNET_KEY

```

kirill@ubuntu-cass:~/airflow-practice-05$ echo -e "AIRFLOW_UID=$(id -u)" > .env
kirill@ubuntu-cass:~/airflow-practice-05$ cat .env
AIRFLOW_UID=1000
kirill@ubuntu-cass:~/airflow-practice-05$

```

Рисунок 5 — вывод cat .env

В docker-compose.yaml были добавлены:

- AIRFLOW__CORE__TEST_CONNECTION: 'Enabled'
- _PIP_ADDITIONAL_REQUIREMENTS: apache-airflow-providers-ssh
apache-airflow-providers-postgres
- AIRFLOW__CORE__LOAD_EXAMPLES: 'false'

```

kirill@ubuntu-cass:~/airflow-practice-05$ kirill@ubuntu-cass:~/airflow-practice-05$ grep -n "TEST_CONNECTION\|_PIP_ADDIT
IONAL_REQUIREMENTS\|LOAD_EXAMPLES" docker-compose.yaml
38:# _PIP_ADDITIONAL_REQUIREMENTS - Additional PIP requirements to add when starting all containers.
65: AIRFLOW__CORE__LOAD_EXAMPLES: 'false'
66: AIRFLOW__CORE__TEST_CONNECTION: 'Enabled'
67: _PIP_ADDITIONAL_REQUIREMENTS: apache-airflow-providers-ssh apache-airflow-providers-postgres
76: # WARNING: Use _PIP_ADDITIONAL_REQUIREMENTS option ONLY for a quick checks
78: _PIP_ADDITIONAL_REQUIREMENTS: ${_PIP_ADDITIONAL_REQUIREMENTS:-}
298: _PIP_ADDITIONAL_REQUIREMENTS: ''
kirill@ubuntu-cass:~/airflow-practice-05$

```

Рисунок 6 — вывод grep с изменёнными строками

Инициализация выполнена командой docker compose up airflow-init. Создана база метаданных, пользователь airflow/airflow.

```

kirill@ubuntu-cass:~/airflow-practice-05$ docker compose up airflow-init
[+] up 32/32
[+] Image redis:7.2-bookworm Pulled 27.8s
[+] Image postgres:16 Pulled 39.1s
[+] Network airflow-practice-05_default Created 0.2s
[+] Volume airflow-practice-05_postgres-db-volume Created 0.0s
[+] Container airflow-practice-05-redis-1 Created 1.9s
[+] Container airflow-practice-05-postgres-1 Created 1.9s
[+] Container airflow-practice-05-airflow-init-1 Created 0.3s
Attaching to airflow-init-1
Container airflow-practice-05-redis-1 Waiting
Container airflow-practice-05-postgres-1 Waiting
Container airflow-practice-05-redis-1 Healthy
Container airflow-practice-05-postgres-1 Healthy

```

Рисунок 7 — вывод airflow-init

```

d. Command not found
kirill@ubuntu-cass:~/airflow-practice-05$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CREATED        STATUS
airflow-practice-05-postgres-1     postgres:16                        "docker-entrypoint.s..."           postgres   3 minutes ago  Up 2 minutes
healthy) 5432/tcp
airflow-practice-05-redis-1        redis:7.2-bookworm                "docker-entrypoint.s..."           redis      3 minutes ago  Up 2 minutes
healthy) 6379/tcp
kirill@ubuntu-cass:~/airflow-practice-05$

```

Рисунок 8 — вывод docker compose ps (postgres, redis — healthy)

Airflow запущен в фоне командой docker compose up -d.

```

[+] up 8/8ntu-cass:~/airflow-practice-05$ docker compose ps
Container airflow-practice-05-redis-1           Healthy 36.3s
Container airflow-practice-05-postgres-1         Healthy 36.3s
Container airflow-practice-05-airflow-apiserver-1 Healthy 64.1s
Container airflow-practice-05-airflow-scheduler-1 Started 37.0s
Container airflow-practice-05-airflow-dag-processor-1 Started 36.0s
Container airflow-practice-05-airflow-triggerer-1 Started 32.7s
Container airflow-practice-05-airflow-worker-1    Started 64.0s
Container airflow-practice-05-airflow-init-1       Exited 34.8s
kirill@ubuntu-cass:~/airflow-practice-05$ docker compose ps
NAME                                IMAGE                                COMMAND                                SERVICE    CR
EATED                                STATUS                                PORTS                                COMMAND                                SERVICE    CR
airflow-practice-05-airflow-apiserver-1     apache/airflow:3.2.1                "/usr/bin/dumb-init ..."           airflow-apiserver    Ab
out a minute ago Up 44 seconds (healthy) 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
airflow-practice-05-airflow-dag-processor-1  apache/airflow:3.2.1                "/usr/bin/dumb-init ..."           airflow-dag-processor Ab
out a minute ago Up 41 seconds (healthy) 8080/tcp
airflow-practice-05-airflow-scheduler-1     apache/airflow:3.2.1                "/usr/bin/dumb-init ..."           airflow-scheduler    Ab
out a minute ago Up 41 seconds (healthy) 8080/tcp
airflow-practice-05-airflow-triggerer-1     apache/airflow:3.2.1                "/usr/bin/dumb-init ..."           airflow-triggerer     Ab
out a minute ago Up 45 seconds (healthy) 8080/tcp
airflow-practice-05-airflow-worker-1        apache/airflow:3.2.1                "/usr/bin/dumb-init ..."           airflow-worker        Ab
out a minute ago Up 13 seconds (health: starting) 8080/tcp
airflow-practice-05-postgres-1              postgres:16                        "docker-entrypoint.s..."           postgres             4
minutes ago Up 4 minutes (healthy) 5432/tcp
airflow-practice-05-redis-1                 redis:7.2-bookworm                "docker-entrypoint.s..."           redis                 4
minutes ago Up 4 minutes (healthy) 6379/tcp
kirill@ubuntu-cass:~/airflow-practice-05$

```

Рисунок 9 — вывод docker compose ps (7 контейнеров, все healthy)

```

kirill@ubuntu-cass:~/airflow-practice-05$ ss -tlnp | grep 8080
LISTEN 0      4096      0.0.0.0:8080      0.0.0.0:*
LISTEN 0      4096      [::]:8080        [::]:*
kirill@ubuntu-cass:~/airflow-practice-05$

```

Рисунок 10 — вывод ss -tlnp | grep 8080 (порт 8080 слушается)

Веб-интерфейс доступен по адресу <http://192.168.58.110:8080>. Логин airflow, пароль airflow.

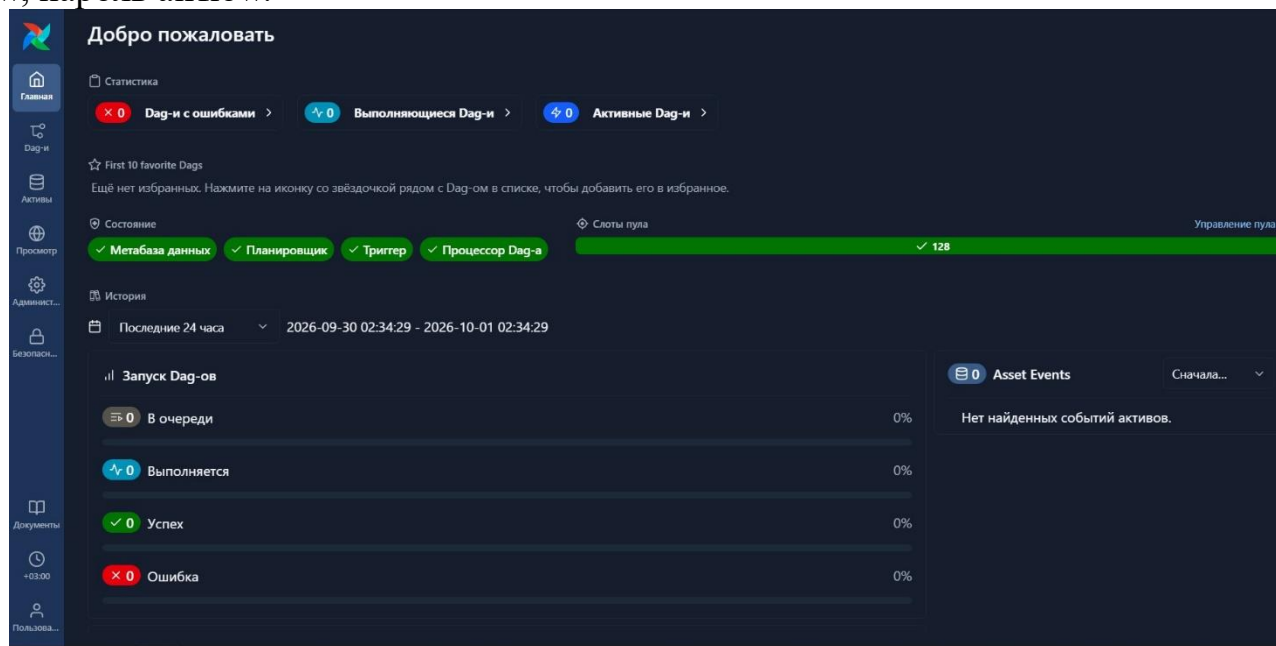


Рисунок 11 — страница входа Airflow

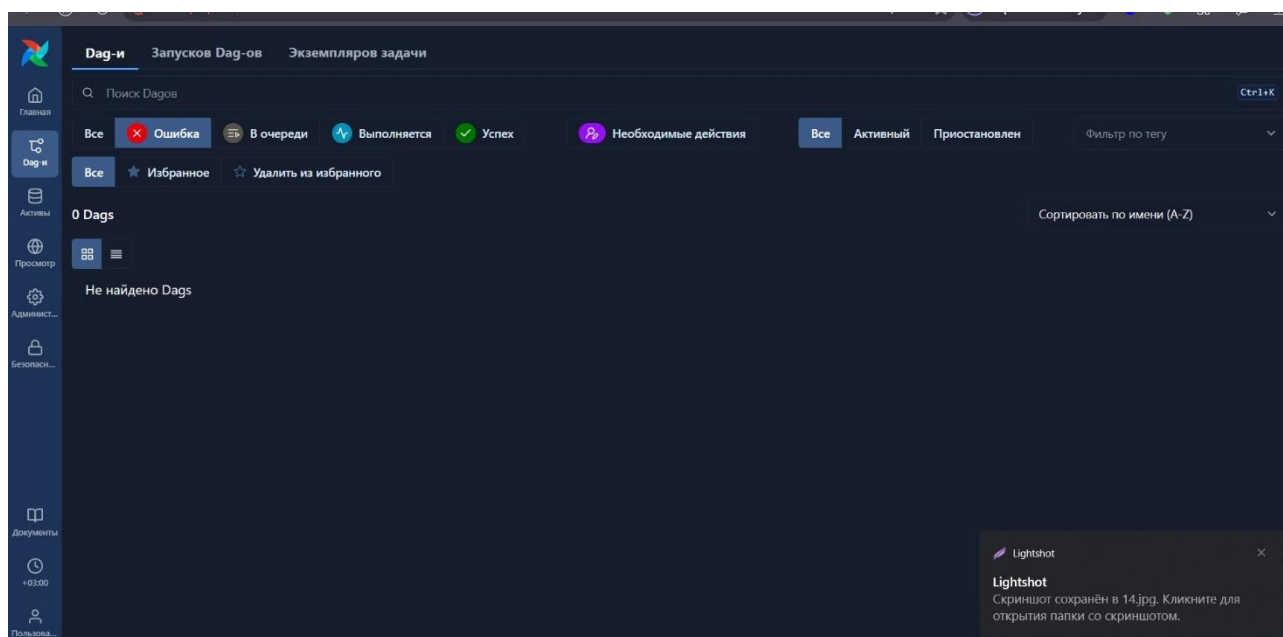


Рисунок 12 — главная страница Airflow.

Создано SSH-подключение ssh_adh_m1:
Type: SSH
Host: 192.168.58.101
Login: user
Password: 123123
Port: 22

Добавить соединение

ID соединения * ssh_adh_m1

Тип соединения * SSH

Отсутствует тип соединения? Убедитесь, что вы установили соответствующий пакет Airflow Providers.

Стандартные поля

Описание

Хост 192.168.58.101

Username user

Пароль

Порт 22

Дополнительные поля JSON

Рисунок 13 — форма создания SSH-соединения

ID соединения	Тип соединения	Описание	Хост	Порт
ssh_adh_m1	ssh		192.168.58.101	22

Тестирование соединения успешно
Connection successfully tested

Рисунок 14 — тест SSH-соединения (успешно)

Создано PostgreSQL-подключение postgres_olist:

Type: Postgres

Host: 192.168.58.110

Login: postgres

Password: secret

Port: 5432

Database: brazilian_commerce

Добавить соединение

ID соединения *

Тип соединения * Postgres

Отсутствует тип соединения? Убедитесь, что вы установили соответствующий пакет Airflow Providers.

Стандартные поля

Описание	
Хост	192.168.58.110
Логин	postgres
Пароль	
Порт	5432
Database	brazilian_commerce

Рисунок 15 — форма создания PostgreSQL-соединения

2 Connections

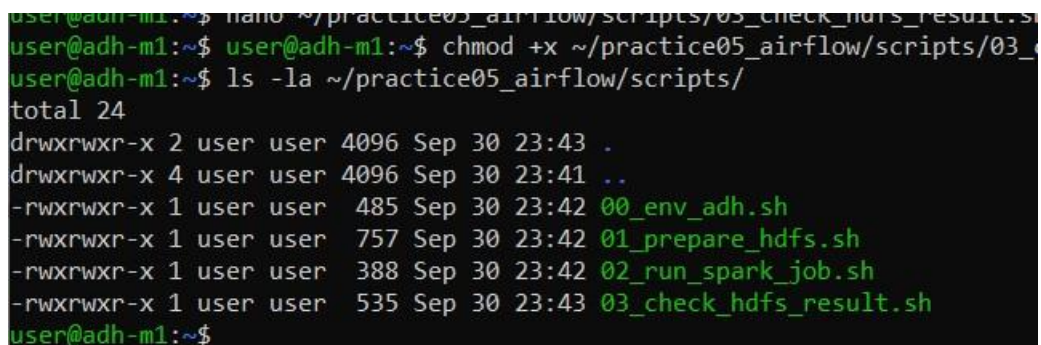
☐	ID соединения	Тип соединения	Описание	Хост	Порт	
☐	postgres_olist	postgres		192.168.58.110	5432	
☐	ssh_adh_m1	ssh		192.168.58.101	22	

Тестирование соединения успешно
Connection successfully tested

Рисунок 16 — тест PostgreSQL-соединения (успешно)

На узле adh-m1 созданы shell-скрипты для оркестрации:

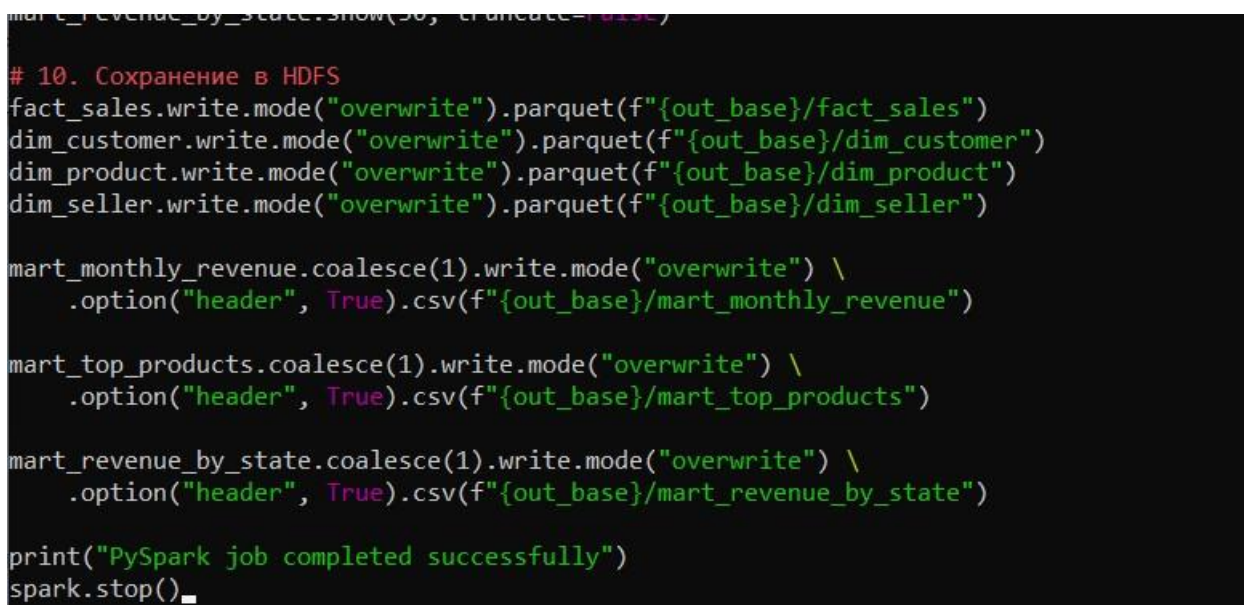
- 00_env_adh.sh — переменные окружения.
- 01_prepare_hdfs.sh — подготовка HDFS.
- 02_run_spark_job.sh — запуск PySpark.
- 03_check_hdfs_result.sh — проверка HDFS.
- 04_prepare_csv_for_adb.sh — выгрузка CSV из HDFS.
- 05_load_marts_to_adb.sh — загрузка CSV в PostgreSQL.



```
user@adh-m1:~$ nano ~/practice05_airflow/scripts/03_check_hdfs_result.sh
user@adh-m1:~$ user@adh-m1:~$ chmod +x ~/practice05_airflow/scripts/03_
user@adh-m1:~$ ls -la ~/practice05_airflow/scripts/
total 24
drwxrwxr-x 2 user user 4096 Sep 30 23:43 .
drwxrwxr-x 4 user user 4096 Sep 30 23:41 ..
-rwxrwxr-x 1 user user 485 Sep 30 23:42 00_env_adh.sh
-rwxrwxr-x 1 user user 757 Sep 30 23:42 01_prepare_hdfs.sh
-rwxrwxr-x 1 user user 388 Sep 30 23:42 02_run_spark_job.sh
-rwxrwxr-x 1 user user 535 Sep 30 23:43 03_check_hdfs_result.sh
user@adh-m1:~$
```

Рисунок 17 — вывод `ls -la ~/practice05_airflow/scripts/` (6 скриптов, все executable)

Создано PySpark-приложение `lab5_hdfs_to_greenplum.py`. Приложение читает CSV из HDFS, выполняет JOIN таблиц Olist, очистку, формирует измерения и таблицу фактов, рассчитывает витрины `mart_monthly_revenue`, `mart_top_products`, `mart_revenue_by_state`.



```
# 10. Сохранение в HDFS
fact_sales.write.mode("overwrite").parquet(f"{out_base}/fact_sales")
dim_customer.write.mode("overwrite").parquet(f"{out_base}/dim_customer")
dim_product.write.mode("overwrite").parquet(f"{out_base}/dim_product")
dim_seller.write.mode("overwrite").parquet(f"{out_base}/dim_seller")

mart_monthly_revenue.coalesce(1).write.mode("overwrite") \
    .option("header", True).csv(f"{out_base}/mart_monthly_revenue")

mart_top_products.coalesce(1).write.mode("overwrite") \
    .option("header", True).csv(f"{out_base}/mart_top_products")

mart_revenue_by_state.coalesce(1).write.mode("overwrite") \
    .option("header", True).csv(f"{out_base}/mart_revenue_by_state")

print("PySpark job completed successfully")
spark.stop()
```

Рисунок 18 — фрагмент `lab5_hdfs_to_greenplum.py` в nano

Созданы SQL-файлы для PostgreSQL:

- 01_create_schema.sql — создание схемы `olist_demo`.

- 02_create_tables.sql — создание таблиц mart_*.
- 03_check_tables.sql — проверка количества строк.
- 04_analyze_tables.sql — сбор статистики.

```
kirill@ubuntu-cass:~/airflow-practice-05$ kirill@ubuntu-cass:~/airflow-practice-05$ nano ~/airflow-practice-05/dags/sql/02_create_tables.sql
kirill@ubuntu-cass:~/airflow-practice-05$ kirill@ubuntu-cass:~/airflow-practice-05$ nano ~/airflow-practice-05/dags/sql/03_check_tables.sql
kirill@ubuntu-cass:~/airflow-practice-05$ kirill@ubuntu-cass:~/airflow-practice-05$ nano ~/airflow-practice-05/dags/sql/04_analyze_tables.sql
kirill@ubuntu-cass:~/airflow-practice-05$ kirill@ubuntu-cass:~/airflow-practice-05$ ls -la ~/airflow-practice-05/dags/sql/
total 24
drwxrwxr-x 2 kirill kirill 4096 Sep 30 23:47 .
drwxrwxr-x 3 kirill root  4096 Sep 30 23:46 ..
-rw-rw-r-- 1 kirill kirill  40 Sep 30 23:46 01_create_schema.sql
-rw-rw-r-- 1 kirill kirill 494 Sep 30 23:46 02_create_tables.sql
-rw-rw-r-- 1 kirill kirill 275 Sep 30 23:47 03_check_tables.sql
-rw-rw-r-- 1 kirill kirill 120 Sep 30 23:47 04_analyze_tables.sql
kirill@ubuntu-cass:~/airflow-practice-05$
```

Рисунок 19 — вывод `ls -la ~/airflow-practice-05/dags/sql/` (4 файла)

Создан DAG `practice_05_adh_ssh_adb_copy` с 11 задачами:

- start, finish — EmptyOperator.
- prepare_hdfs_on_adh, run_spark_job_on_adh,
check_hdfs_result_on_adh, prepare_csv_for_adb_on_adh,
load_marts_to_adb_via_psql_copy — SSHOperator.
- create_greenplum_schema, create_greenplum_tables,
check_greenplum_result, analyze_greenplum_tables —
SQLExecuteQueryOperator.

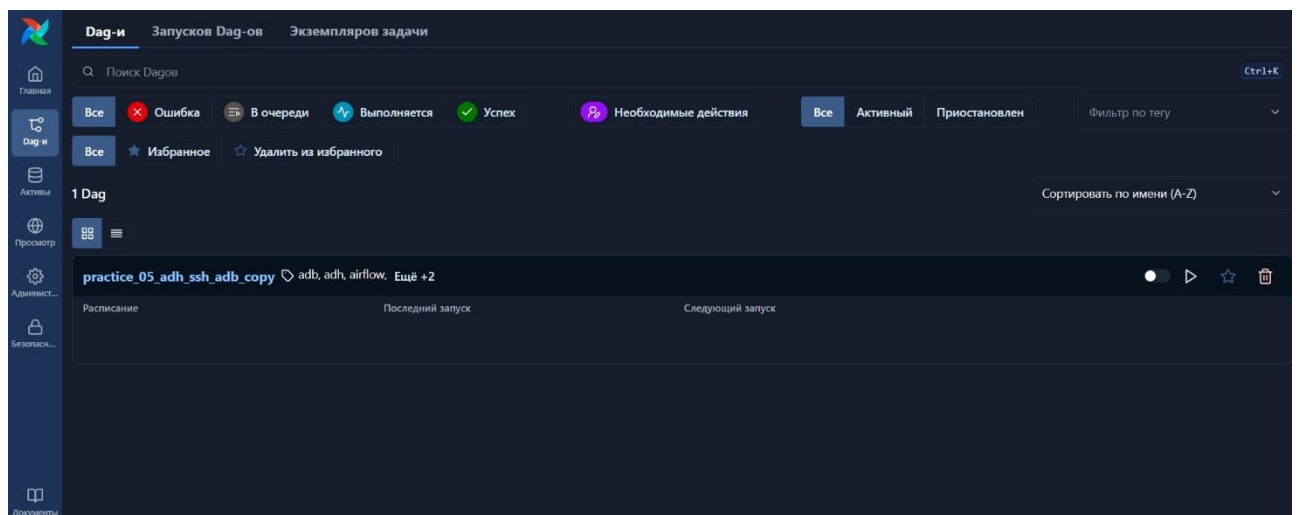


Рисунок 20 — фрагмент DAG в папо

DAG запущен вручную из веб-интерфейса Airflow. Все 11 задач выполнены успешно. Длительность запуска — 2 минуты 47 секунд.

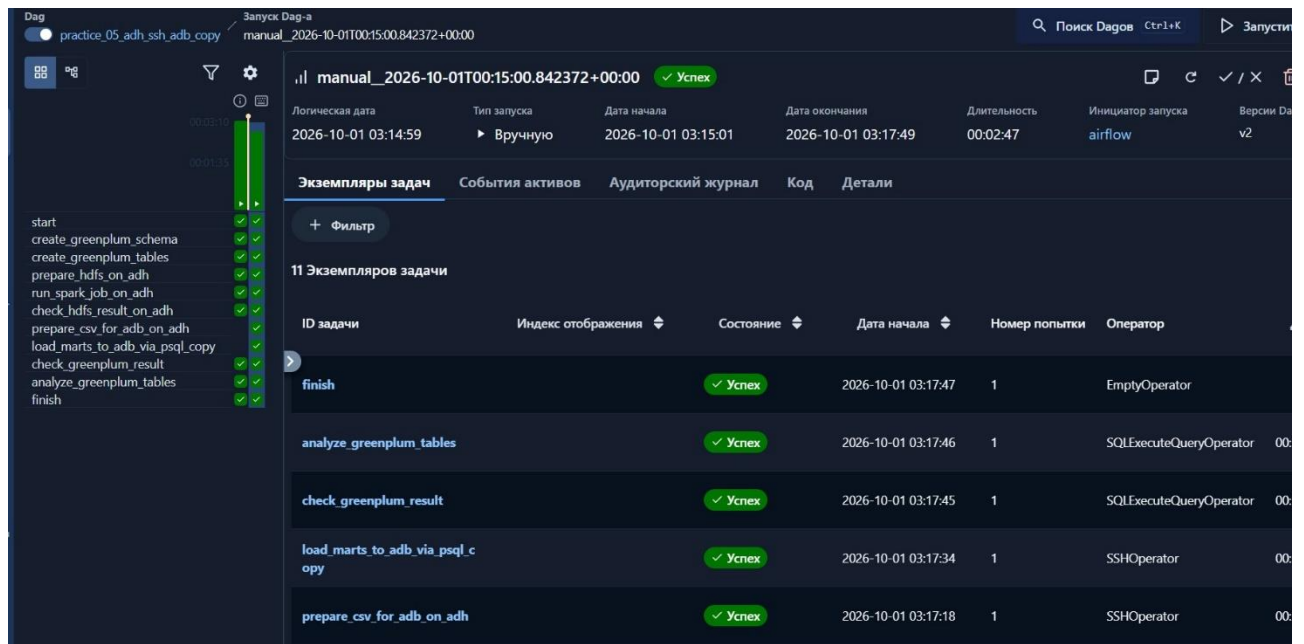


Рисунок 21 — Grid view DAG (все задачи зелёные)

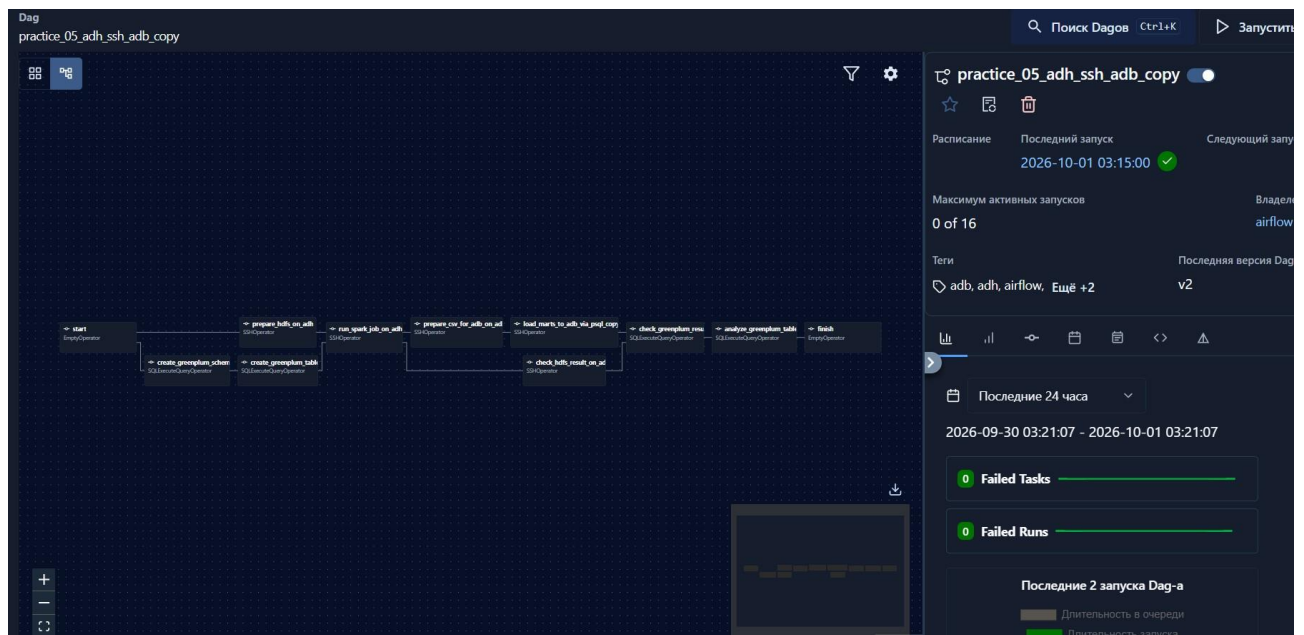


Рисунок 22 — Graph view DAG (граф задач)

Когда	Событие	Пользователь	Дополнительно	ID запуска	ID задачи	Инд
2026-10-01 03:17:47	success	airflow	>{...}	manual_2026-10-01T00:15:00.842372+00:00	analyze_greenplum_tables	-1
2026-10-01 03:17:46	running	airflow	>{...}	manual_2026-10-01T00:15:00.842372+00:00	analyze_greenplum_tables	-1
2026-10-01 03:17:46	success	airflow	>{...}	manual_2026-10-01T00:15:00.842372+00:00	check_greenplum_result	-1
2026-10-01 03:17:45	running	airflow	>{...}	manual_2026-10-01T00:15:00.842372+00:00	check_greenplum_result	-1
2026-10-01 03:17:44	success	airflow	>{...}	manual_2026-10-01T00:15:00.842372+00:00	check_hdfs_result_on_adh	-1

Рисунок 23 — аудиторский журнал Airflow (события success)

Витрины в PostgreSQL заполнены:

mart_monthly_revenue — 23 строки.

mart_top_products — 10 строк.

mart_revenue_by_state — 27 строк.

```
kirill@ubuntu-cass:~/airflow-practice-05$ psql -h 192.168.58.110 -U postgres -d brazilian_commerce -c "
> SET search_path TO olist_demo;
> SELECT 'mart_monthly_revenue' AS table_name, COUNT(*) FROM mart_monthly_revenue
> UNION ALL
> SELECT 'mart_top_products', COUNT(*) FROM mart_top_products
> UNION ALL
> SELECT 'mart_revenue_by_state', COUNT(*) FROM mart_revenue_by_state;
> "
Password for user postgres:
SET
 table_name | count
-----+-----
 mart_monthly_revenue | 23
 mart_top_products | 10
 mart_revenue_by_state | 27
(3 rows)
```

Рисунок 24 — счётчики витрин (23 / 10 / 27)

```

kirill@ubuntu-cass:~/airflow-practice-05$ psql -h 192.168.58.110 -U postgres -d brazilian_commerce -c "
> SET search_path TO olist_demo;
> SELECT * FROM mart_monthly_revenue ORDER BY month_dt LIMIT 10;
> "
Password for user postgres:
SET

```

month_dt	revenue	orders_cnt
2016-09-01	143.46	1
2016-10-01	46490.66	265
2016-12-01	19.62	1
2017-01-01	127482.37	750
2017-02-01	271239.32	1653
2017-03-01	414330.95	2546
2017-04-01	390812.40	2303
2017-05-01	566851.40	3546
2017-06-01	490050.37	3135
2017-07-01	566299.08	3872

(10 rows)

Рисунок 25 — данные mart_monthly_revenue (первые 10 месяцев)

```

kirill@ubuntu-cass:~/airflow-practice-05$ psql -h 192.168.58.110 -U postgres -d brazilian_commerce -c "
> SET search_path TO olist_demo;
> SELECT * FROM mart_top_products ORDER BY revenue DESC LIMIT 5;
> "
Password for user postgres:
SET

```

product_category	revenue	items_cnt
health_beauty	1412089.53	9465
watches_gifts	1264333.12	5859
bed_bath_table	1225209.26	10953
sports_leisure	1118256.91	8431
computers_accessories	1032723.77	7644

(5 rows)

Рисунок 26 — данные mart_top_products (топ-5)

```

kirill@ubuntu-cass:~/airflow-practice-05$ psql -h 192.168.58.110 -U postgres -d brazilian_commerce -c "
> SET search_path TO olist_demo;
> SELECT * FROM mart_revenue_by_state ORDER BY revenue DESC LIMIT 10;
> "
Password for user postgres:
SET

```

customer_state	revenue	orders_cnt
SP	5769703.15	40501
RJ	2055401.57	12350
MG	1818891.67	11354
RS	861472.79	5345
PR	781708.80	4923
SC	595127.78	3546
BA	591137.81	3256
DF	346123.35	2080
GO	334212.35	1957
ES	317657.93	1995

(10 rows)

Рисунок 27 — данные mart_revenue_by_state (топ-10 штатов)

ВЫВОДЫ

В ходе практической работы были сформированы навыки оркестрации распределённых вычислений с применением Apache Airflow.

Были выполнены:

- развёртывание Apache Airflow 3.2.1 в Docker Compose на ВМ ubuntu-cass;
- настройка SSH-подключения Airflow к узлу ADH;
- настройка PostgreSQL-подключения к аналитической СУБД;
- подготовка shell-скриптов и PySpark-приложения на узле ADH;
- создание SQL-файлов для работы с PostgreSQL;
- создание DAG practice_05_adh_ssh_adb_cory из 11 задач;
- запуск DAG и проверка результатов.

В результате выполнения DAG:

- PySpark-приложение выполнено на кластере ADH, результаты сохранены в HDFS;
- аналитические витрины mart_monthly_revenue, mart_top_products, mart_revenue_by_state загружены в PostgreSQL;
- проверена работа всех сервисов: Airflow, SSH, PostgreSQL, Spark.
- Полученные результаты подтверждают корректность разработанного DAG и готовность инфраструктуры к оркестрации распределённых вычислений.